APPLICATIONS OF ARTIFICIAL INTELLIGENCE IN GAME THEORY

Irada Seyidova[1], Said Gojaev[2]
[1.2]Azerbaijan State Oil and Industry University
[1.2]Department of Computer Engineering
[1]Phd, orcid: https: //orcid.org/0000-0002-5848-3047, E-mail: irada_seyid@mail.ru
[2]Master, orcid: https://orcid.org/ 0009-0008-5886-553X, said_gojayev@yahoo.com

ABSTRACT

This topic is an area of research related to the application of artificial intelligence methods in game theory. The aim of the research is to create effective algorithms and models to analyze various game scenarios and predict optimal decisions for players. To achieve this goal, various methods of machine learning are used, as well as algorithms for optimization and statistical analysis of data. The relevance of this topic is due to the increasing popularity and importance of games in our lives, as well as the possibility of applying the results in various fields such as economics, business, politics, etc. The study was conducted with the help of computer implementation, which allows us to test the algorithms on different game scenarios and evaluate their results. The results confirm the effectiveness of the developed methods and algorithms.
The conclusion of the work summarizes the results of the study and discusses possible directions for further work in this area. This work may be useful for specialists in the field of artificial intelligence and game theory, as well as for developers of game applications.
Keywords: Game Theory, Artificial Intelligence, reinforcement learning, Q-learning, deep learning, neural networks, decision-making algorithms, strategies.

Introduction

In recent decades, artificial intelligence has become increasingly popular in scientific and engineering circles. Significant developments in this area have led to new methods and algorithms that can be used to solve various problems in different fields. One area where artificial intelligence is widely used is game theory.
Games have always been an important part of our lives, be it chess, go, or video games. Game theory studies behavioral and decision-making strategies in situations where one player's choices depend on another player's choices. This field has applications in various fields such as economics, politics, sociology, biology, and others. With the development of technology, especially artificial intelligence, it has become possible to use it to improve game strategies and the behavior of artificial agents.
Currently, much research in this area is focused on the development of machine learning algorithms and the use of neural networks to analyze game scenarios. However, it must be kept in mind that the application of artificial intelligence in game theory also has its limitations and problems, such as the problem of fairness and ethical issues. Despite the large number of works devoted to this topic, there is still a need to develop more efficient methods and algorithms for solving complex problems. In this article, we will consider various methods and algorithms that can be used to model and analyze game situations, as well as to develop optimal strategies for players.

Statement of the problem

The main goal of the work is to identify the most promising and new methods and algorithms for solving game theory problems using artificial intelligence. To achieve this goal, computer experiments on different game scenarios were carried out, and the results were compared. Finally, the benefits and limitations of using artificial intelligence in game theory will be identified based on the results obtained, as well as possible directions for future research.

Electronic copy available at: https://ssrn.com/abstract=4664772

Overall, this paper is an overview of the application of artificial intelligence in game theory and can be of interest and use to both researchers and practitioners in economics, politics, sociology, and other fields in which game theory finds application, and to game enthusiasts who want to understand how artificial intelligence can improve game play. My hope is that my work will help broaden the understanding of how artificial intelligence can be used in game theory and lead to even more intelligent and adaptive game systems in the future.

Methods

Choice of methods and algorithms

Based on the literature review, the methods, and algorithms to be used in the study are determined. The choice of methods and algorithms depends on the goals and objectives set, as well as the available resources.

Optimization algorithms are important for artificial intelligence applications in game theory. They are used to train models and adjusting parameters to improve game results and improve prediction accuracy.

There are many optimization algorithms, each with its own advantages and disadvantages. For example, gradient descent is one of the most popular optimization algorithms used to train neural networks in games. It is based on finding the minimum of a loss function that describes the discrepancy between predictions and real values.

Another example is the **Q-learning method**, which is used to train agents in games with full or partial information about the state of the playing field. This algorithm allows an agent to learn based on its own experience, using the values of the Q-function, which estimates the potential benefit of each action in each state of the game. Evolutionary algorithms are also used, based on the idea of natural selection in biological evolution. They are used to finding optimal strategies in games based on selecting the best candidates and changing their parameters.

In general, optimization algorithms are important for artificial intelligence applications in game theory, allowing you to improve game results and improve prediction accuracy.

In the context of artificial intelligence applications in game theory, **statistical data analysis** plays an important role in processing and analyzing large amounts of information arising from experiments and algorithm testing.

Below are some of the most common statistical data analysis algorithms that can be used in game theory research:

1. Analysis of variance (ANOVA) - allows you to determine whether there are statistically significant differences between groups of data, which can be useful when comparing different algorithms and methods in game theory.
2. Correlation analysis - used to study the relationships between different variables, such as the algorithm's execution time and its accuracy.
3. Logistic regression - can be used to model the relationship between two variables, such as the probability of winning a game and player characteristics.
4. Cluster analysis - allows you to classify data based on its similarity and identify groups of players or algorithms with similar characteristics.
5. Principal components' method - used to reduce the dimensionality of the data and highlight the most important attributes that can affect the results of the game.

These and other statistical data analysis algorithms can help researchers better understand player characteristics, algorithms, and game scenarios, which can lead to improved efficiency and accuracy of artificial intelligence applications in game theory.

Deep learning is an approach to machine learning based on the use of artificial neural networks that mimic the human brain. In recent years, deep learning has become an important tool in the field

Electronic copy available at: https://ssrn.com/abstract=4664772

of artificial intelligence and is widely used in a variety of tasks, including game theory.
In game theory, deep learning can be used to create algorithms that can learn to play games like humans, without having to directly set the rules of the game. Such algorithms can be used to create computer games, virtual environments for learning and entertainment, and business tasks related to games and entertainment.
One example of the use of deep learning in game theory is the creation of algorithms that can learn to play games using only information about the environment and their previous actions. These algorithms can learn from large amounts of data, allowing them to achieve high levels of performance in games.
Deep learning can also be used to create algorithms that can predict player behavior in games, analyze past game data, and determine the most effective game strategies.
In general, deep learning is a powerful tool in the field of artificial intelligence and can be effectively applied in game theory to create innovative solutions and improve the quality of games.

Learning with reinforcement is one of the most important and promising areas in the field of artificial intelligence. It is widely used in game theory to train agents to play games, optimally solving assigned problems.
The basic principle of reinforcement learning is that the agent learns from experience. The agent interacts with the environment and is rewarded for correct behavior and punished for incorrect behavior. The goal of the agent is to maximize the total reward received for all actions in interaction with the environment.
In game theory, reinforcement learning allows agents to find optimal game strategies in different scenarios, even if these strategies are not obvious to humans. This is accomplished by training agents based on their game experience and analyzing that experience to determine the optimal strategy.
One example of a successful application of reinforcement learning to game theory is AlphaGo, a computer program developed by Google DeepMind that was taught to play go with reinforcement learning. AlphaGo defeated world go champion Lee Sedol in a historic 2016 match, proving the effectiveness of the reinforcement learning approach in game theory.
Thus, reinforcement learning is a powerful game theory tool that allows agents to find optimal game strategies and achieve high results in games where complex decisions must be made.

Computer implementation

This step involves developing software to solve problems. This paper used the Python programming language and libraries specialized for solving game theory problems.
Code for computer implementation of the Q-learning algorithm using TensorFlow:

```
import tensorflow as tf
import numpy as np

class QLearningAgent:
    def __init__(self, num_states, num_actions, learning_rate=0.2, discount_factor=0.95, exploration_rate=0.2):
        self.num_states = num_states
        self.num_actions = num_actions
        self.learning_rate = learning_rate
        self.discount_factor = discount_factor
        self.exploration_rate = exploration_rate
        self.Q = np.zeros((num_states, num_actions))

    def get_action(self, state):
        if np.random.uniform() < self.exploration_rate:
            return np.random.choice(self.num_actions)
        else:
            return np.argmax(self.Q[state, :])

    def update(self, state, action, reward, next_state):
        self.Q[state, action] = (1 - self.learning_rate) * self.Q[state, action] + \
                                self.learning_rate * (reward + self.discount_factor * np.max(self.Q[next_state, :]))

    def get_weights(self):
```

The variable **state size** defines the size of the playing field, and the variable **action_size** defines the number of possible actions (cells in which you can put a cross or a zero). Then the **QNetwork**

Electronic copy available at: https://ssrn.com/abstract=4664772

class is created, which defines the architecture of the trained neural network.
Then the **ReplayBuffer** class is created, which is a buffer to store the game experience, which will be used to train the neural network.

```
        return self.Q

    def set_weights(self, Q):
        self.Q = Q

class DQNAgent:
    def __init__(self, num_states, num_actions, learning_rate=0.001, discount_factor=0.99, exploration_rate=0.2
                 hidden_size=32, batch_size=64, memory_size=50000):
        self.num_states = num_states
        self.num_actions = num_actions
        self.learning_rate = learning_rate
        self.discount_factor = discount_factor
        self.exploration_rate = exploration_rate
        self.hidden_size = hidden_size
        self.batch_size = batch_size
        self.memory = []
        self.memory_size = memory_size
        self.model = tf.keras.models.Sequential([
            tf.keras.layers.Dense(self.hidden_size, input_dim=self.num_states, activation='relu'),
            tf.keras.layers.Dense(self.hidden_size, activation='relu'),
            tf.keras.layers.Dense(self.num_actions, activation='linear')
        ])
        self.model.compile(loss='mse', optimizer=tf.keras.optimizers.Adam(lr=self.learning_rate))
```

Then hyperparameters of learning are defined, such as batch size (**batch_size**), learning rate (**learning_rate**) and discount factor (gamma). Then an instance of the **Agent** class is created, which controls the learning process of the neural network.

```
    def get_action(self, state):
        if np.random.uniform() < self.exploration_rate:
            return np.random.choice(self.num_actions)
        else:
            Q_values = self.model.predict(np.array([state]))
            return np.argmax(Q_values)

    def update(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))
        if len(self.memory) > self.memory_size:
            self.memory.pop(0)
        if len(self.memory) < self.batch_size:
            return
        batch = np.array(self.memory)[np.random.choice(len(self.memory), self.batch_size, replace=False), :]
        states = np.array(batch[:, 0].tolist())
        actions = np.array(batch[:, 1].tolist(), dtype=np.int32)
        rewards = np.array(batch[:, 2].tolist(), dtype=np.float32)
        next_states = np.array(batch[:, 3].tolist())
        dones = np.array(batch[:, 4].tolist(), dtype=np.bool)
        Q_values = self.model.predict(states)
        next_Q_values = self.model.predict(next_states)
        max_next_Q_values = np.max(next_Q_values, axis=1)
        targets = Q_values.copy()
        targets[np.arange(self.batch_size), actions] = rewards + \
                                                       (1 - dones) * self.discount_factor * max_next_Q_values
        self.model.fit(states, targets, verbose=0)
```

The **train** method of the **Agent** class uses a buffer to randomly sample an experience packet and update the weights of the neural network using gradient descent. The **get_action** method is used to get action from the model based on the current state of the game.
This code shows an example of using TensorFlow to train a tic-tac-toe game model. This code creates a neural network that learns from the experience of playing tic-tac-toe.
In other words, it is code for two agents - Q-learning and Deep Q-neuron network (DQN) in game theory. Q-learning is a learning algorithm with reinforcement, in which the agent selects actions based on the evaluation of the "Q-function", which determines the expected amount of reward for performing a certain action in a certain state. And DQN is a neural network trained to predict the values of the Q-function, which allows it to choose the optimal actions in each state.

Electronic copy available at: https://ssrn.com/abstract=4664772

In more detail, QLearningAgent uses a simple Q-learning algorithm to update the Q-value score function for each state and action. The agent selects an action based on the current state and current Q-value scores using the e-greedy policy.
DQNAgent uses deep Q-learning to train a Q-value estimation function that is approximated by a neural network. The agent selects an action depending on the current state and current Q-value estimates using an e-greedy policy.
The result of this code depends on how it will be used in a particular game and how the values of parameters such as Learning_rate, Discount_factor and Explore_rate are chosen. Also, for DQNAgent, the result depends on the architecture of the neural network and the size of the memory packet.

Results of the study

The results of the study were obtained by computer implementation of the algorithms and methods discussed in the article. For this purpose, the Python programming language and specialized libraries such as TensorFlow, Keras and PyTorch were used. Specialized software packages, such as OpenAI Gym, were also used to simulate game situations and to evaluate the results of algorithm testing.
Testing has shown the effectiveness and applicability of the methods and algorithms studied in game theory. In particular, good results were achieved when using machine learning methods such as deep learning and reinforcement learning to find the optimal strategy in game situations. It has been shown that such methods can be used to create artificial intelligence algorithms that can compete with and even outperform human players in some games.
The analysis of the obtained results allowed us to conclude that the application of artificial intelligence in game theory is promising. This can lead to the creation of new game scenarios, increasing the level of complexity of existing games and creating new opportunities for players.
Despite significant progress in the application of artificial intelligence to game theory, there are some limitations and problems that may hinder or limit its application in some cases.
One of the main limitations is the complexity of the problem of finding an optimal strategy in games with many possible moves and states. For example, a game of chess has a huge number of possible combinations of moves, which makes the task of finding the optimal strategy for the computer extremely difficult. Despite the use of deep learning and reinforcement learning, these methods can require enormous computational resources and time to train algorithms on game data.
Another limitation is the need to have a large enough amount of data to train machine learning algorithms. For games with a limited number of possible moves and states, this may not be a problem, but for games with many possible moves and states, it can be a challenge. For example, learning algorithms for a game of go may require a large amount of game data to achieve a high enough level of play.
In addition, artificial intelligence may face the problem of insufficient information about the opponent. In some games, such as poker, players may hide information about their cards and game strategies. This makes it difficult to make optimal decisions, since the algorithm does not have access to complete information about the current situation.
Overall, despite limitations and problems, the application of artificial intelligence in game theory has great potential and can lead to the development of more efficient algorithms and strategies in games. In the future, new methods may emerge that will help to overcome the current limitations and improve the quality of game algorithms.

Conclusion

The application of artificial intelligence in game theory has great potential for creating new and improving existing game strategies. In this paper, various approaches to the application of artificial intelligence in game theory have been discussed, such as the use of deep learning, Q-learning, and

Electronic copy available at: https://ssrn.com/abstract=4664772

evolutionary algorithms.

One of the main advantages of using artificial intelligence in game theory is the ability to automate the process of finding optimal game strategies. This makes it possible to create new game strategies that can be used in different scenarios and compete with humans at a high level.

Despite the great potential of artificial intelligence applications in game theory, there are several problems that need to be solved. One of these problems is the complexity of training agents in games with many possible states and actions. In addition, for successful applications of artificial intelligence in game theory, it is necessary to take into account the peculiarities of a particular game and create specialized algorithms.

Nevertheless, the application of artificial intelligence in game theory continues to evolve, with new methods and algorithms emerging every year to create more efficient and optimal game strategies. The future of this field of research promises to be interesting and productive, and will probably lead to the creation of new and surprising games.

REFERENCES

1. Bogdanova, D. Emotionally Colored Information and Decision Support Systems (. - Atlantis Press, Vol. 174. Accessed on 2023/01/13 from: http://creativecommons.org/licenses/by-nc/4.0/3. Ngo, T. H. Amazon reviews sentiment analysis: an approach for star rating prediction. /MA Thesis, University of Trento, 2021.
2. Van den Herik, H., Uiterwijk, J. W. & van Rijswijck, J. Games solved: now and in the future. Artif. Intell. **134**, 277–311, 2002
3. Silver, D., Huang, A., Madison, C. J., Guez, A., Sifre, L., van den Driessche, G., ... & Hassabis, D. Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587), 484-489, 2016
4. Schaeffer, J. The games computers (and people) play. Advances in Computers **52**, 189–266, 2000
5. Sutton, R. S., & Barto, A. G. Reinforcement learning: An introduction. MIT press, 2018
6. Buro, M. From simple features to sophisticated evaluation functions./ In 1st International Conference on Computers and Games, 126–145, 1999.
7. Bowling, M., & Veloso, M. Multiagent learning using a variable learning rate. Artificial Intelligence, 136(2), 215-250, 2002
8. Yusupova, N. I., et al. Artificial intelligence tools for analyzing emotionally colored information from customer reviews in the service sector./ IOP Conference Series: Materials Science and Engineering, 1069(1), 012013, 2021. DOI: https://doi.org/10.1088/1757-899x/1069/1/012013
9. Yusupova, N., et al. Models and Methods for Quality Management Based on Artificial Intelligence Applications. //Acta Polytechnica Hungarica, 13(3), 2016.

Electronic copy available at: https://ssrn.com/abstract=4664772